

Python for Web Developers: Achievement 2 Learning Journal

Pre-work

- 1. Was your study routine effective during Achievement 1? If not, what will you do differently during Achievement 2?**

I tend to be a little loose with my study routine. I try to get an exercise reading and the chapter quiz done each day every couple of days, and if time allows, I'll either get started on taking notes from the reading or doing the task; if time does not allow, I will try to get both done the next day. If I fall behind during the week, I will try to work on the remaining content over the weekend if I don't already have prior obligations or plans. I often have to go to the chiropractor during the week, at least twice weekly, to continue attempting to treat my back injury, so that eats into some of my time from day to day. Additionally, sometimes I'm in so much pain, I can't sit at the desktop to do coding work since that's where I do all of it. This, coupled with the amount of notes I write per chapter, tends to set me back in my course timeline.

- 2. Reflect on your learning and project work for Achievement 1. What were you most proud of? How will you repeat or build on this in Achievement 2?**

I'm proud of my code structure and commenting skills, and I will continue to improve on both of these through Achievement 2 and beyond. I feel like I've also gotten better at working with ChatGPT when using it to assist with coding.

- 3. What difficulties did you encounter in the last Achievement? How did you deal with them? How could this experience prepare you for difficulties in Achievement 2?**

My main difficulties during last Achievement revolved around applying previous exercise's content to the next exercise's task coding. Sometimes, it was tricky knowing where to start when writing certain things like try-except blocks or the utilization of certain functions like `.join()` or `.split()`. Given more time spent reviewing the chapter readings, my notes, other students' submissions, and working with ChatGPT, I eventually got a better feel for these concepts. If I get stuck with anything in Django, I hope I can reach out to my mentor, other students, or the community online.

Achievement 2: Web Development & Django

Exercise 2.1: Getting Started with Django

- 1. Suppose you're a web developer in a company and need to decide if you'll use vanilla (plain) Python for a project, or a framework like Django instead. What are the advantages and drawbacks of each?**

Vanilla Python is better suited for smaller projects with minimal functionality and allows for more control over every aspect of an app, avoiding the constraints of frameworks like Django. However, development can be slower because the main features of the app, like routing and security, have to be built from scratch, rather than being included with a framework.

Frameworks like Django can be useful for larger, more complex projects as they usually come with built-in features that can handle things like routing, security, functionality, and scalability, cutting down on development time. However, some frameworks, like Django, can have a steep learning curve and be more challenging to work with for those less familiar with them. Additionally, their built-in features can add much additional overhead to the app, resulting in reduced performance. Also, in the case of Django, the app would have to be built the "Django Way", so it would have to follow a specific ruleset to be developed properly.

- 2. In your own words, what is the most significant advantage of Model View Template (MVT) architecture over Model View Controller (MVC) architecture?**

MVT has a significant advantage over MVC because it relies on an integrated templating system. It simplifies rendering HTML content and allows devs to embed content into their templates.

- 3. Now that you've had an introduction to the Django framework, write down three goals you have for yourself and your learning process during this Achievement.**

I would like to learn more about what Django can really do on the backend for an app that uses a database, as well as how it renders content on the frontend. I want to add Django as another tool to my skillset that I can use in future projects. After this achievement, I will be working on finishing the Job Prep course and getting ready to go on the hunt.

Exercise 2.2: Django Project Setup

- 1. Suppose you're in an interview. The interviewer gives you their company's website as an example, asking you to convert the website and its different parts into Django terms. How would you proceed? For this question, you can think about your dream company and look at their website for reference.**

Consider the website cloakbrand.com. The website as a whole would be considered a Django project. Just on its home page, which changes depending on what the currently featured apparel collection is, there are various apps, such as dropdown menus for other collections, categories, and information pages. Additionally, the Cloak logo in the center, as well as the search icon and shopping bag icon, would also be considered apps. Further down the page are a large amount of different featured items for purchase, and each one would also be considered an app. Each of these items would also be tied to a database where each of these items from their respective collections would be stored and fetched from to be displayed on the frontend.

- 2. In your own words, describe the steps you would take to deploy a basic Django application locally on your system.**

To deploy a basic Django app locally, first create a virtual environment and install Django in it. Once the project structure is created, run a migration to create the database of the application. Once everything displays as OK in the terminal, run the server from the terminal, then follow the local host link displayed to view the application in browser to confirm successful deployment.

- 3. Do some research about the Django admin site and write down how you'd use it during your web application development.**

During development of an application using Django, I would be able to use CRUD operations in order to work with data without having to write additional code. I would also be able to observe any changes to the database and test data in it to check that models and views are working correctly. Additionally, I could manage users of the application and their permissions as well as the content that would be posted or generated by them.

Exercise 2.3: Django Models

- 1. Do some research on Django models. In your own words, write down how Django models work and what their benefits are.**

Django models are used to define the structure of tables in a database and behave similarly to Python classes. They allow you to define a class (model) structure for each app in a Django application, as well as their attributes and methods. Benefits of Django models include its use of ORM, ability to make migrations of changes over time, include validation fields to ensure proper field formatting, and an easy-to-access admin interface.

- 2. In your own words, explain why it is crucial to write test cases from the beginning of a project. You can take an example project to explain your answer.**

It's important to write test cases early on in development because they can catch bugs and errors earlier on, saving time down the road. Additionally, they also act as part of the project's documentation. A well-tested project also shows that the code, and its developers, are reliable.

Exercise 2.4: Django Views & Templates

1. **Do some research on Django views. In your own words, use an example to explain how Django views work.**

Django views can be set up as functions or classes, and they receive a web request and return a web response. The view of an app is set up in the `views.py` file of the app directory, where a function is defined to receive the request and return the `HttpResponse`. Then, in the `urls.py` file of the app, the defined function is imported, and its URL pattern path is defined. When the server is run and the URL visited, the `HttpResponse` written in the function will be displayed.

2. **Imagine you're working on a Django web development project, and you anticipate that you'll have to reuse lots of code in various parts of the project. In this scenario, will you use Django function-based views or class-based views, and why?**

Class-based views are preferable in this case because they are implemented as Python classes and can be reused by using object-oriented programming.

3. **Read Django's documentation on the [Django template language](#) and make some notes on its basics.**

A template is a text file that can be used to generate HTML, XML, CSV, etc. They contain variables, which, when the template is evaluated, get replaced with values. They also contain tags, which control the logic of the template. Variables are enclosed in double curly braces and use a filter to modify their value. Tags provide logic like loops and conditionals.

Exercise 2.5: Django MVT Revisited

1. In your own words, explain Django static files and how Django handles them.

In Django, static files consist of images, JavaScript files, CSS files, and anything else that do not change dynamically, like user-generated media. In the settings.py file of a Django application project folder, the STATIC_URL is used when referring to static files located in STATIC_ROOT. A directory for static files can be created at the app level, where all static files can be stored, or within individual apps to only be accessed within that app. As long as the path is specified, and {% load static %} is included at the beginning of template files, Django will be able to access the static files.

2. Look up the following two Django packages on Django's official documentation and/or other trusted sources. Write a brief description of each.

Package	Description
ListView	A page representing a list of objects. When the view executes, the model's records are stored in variable object_list , the individual records of which can be accessed by looping through it.
DetailView	A page representing details of an object. Designed to show detailed information about a specific instance of a model.

3. You're now more than halfway through Achievement 2! Take a moment to reflect on your learning in the course so far. How is it going? What's something you're proud of so far? Is there something you're struggling with? What do you need more practice with? You can use these notes to guide your next mentor call.

I'm becoming more comfortable with working with the Django project directory structure, even though it's still a lot of content to take in. The bookstore application project has been very helpful in learning about and understanding Django applications, and even more helpful in applying this knowledge to the recipe

application. I did, though, have to spend extra time teaching myself how to work with CSS files in the apps, but it went well. I've also continued to have success working with ChatGPT to assist in understanding content and writing coherent code.

Exercise 2.6: User Authentication in Django

- 1. In your own words, write down the importance of incorporating authentication into an application. You can take an example application to explain your answer.**

It's important to incorporate authentication into any application that may contain sensitive data about your users or even if you want your users to maintain their own saved data. In the instance of the recipe app, there isn't likely to be much sensitive data, beyond their user information like password and email. Despite this, to allow users to have their own saved recipes under their individual accounts, and not allow other users to modify their saved data, these views need to be protected. So if a view is protected, a user will have to be authenticated.

- 2. In your own words, explain the steps you should take to create a login for your Django web application.**
 - a. Create the login view that will house the login logic in views.py of the project folder.
 - b. Create the login HTML template that will apply a csrf_token, then write the {{form}} into a <form> tag.
 - c. Add the URL pattern to urls.py at the project folder level.
 - d. Set the login URL in settings.py of the project folder.
- 3. Look up the following three Django functions on Django's official documentation and/or other trusted sources and write a brief description of each.**

Function	Description
authenticate()	Used to verify a set of credentials (username, password), checks them against each authentication backend, and returns a User if the credentials are valid
redirect()	Returns an HttpResponseRedirect to the appropriate URL for the arguments passed (either get_absolute_url() or reverse())

include()	Allows including a template inside the current template (useful for a block of content that is the same for many pages)
------------------	---

Exercise 2.7: Data Analysis and Visualization in Django

1. **Consider your favorite website/application (you can also take CareerFoundry). Think about the various data that your favorite website/application collects. Write down how analyzing the collected data could help the website/application.**

In the instance of CareerFoundry, analyzing user engagement, such as how much time is spent on courses and rate of completion, can help inform decisions on course improvements. They also use feedback in the form of surveys to get insight into student experience directly. They also can view technical metrics like load times and errors to ensure smooth user experience.

2. **Read the [Django official documentation on QuerySet API](#). Note down the different ways in which you can evaluate a QuerySet.**

- a. Iteration: use a for loop to retrieve objects one by one
- b. Slicing: retrieves a subset of objects from database based on slice indices
- c. Pickling: retrieves all objects and serializes them into a specified format
- d. Explicit evaluation methods:
 - i. `list()`: evaluates and returns a list of objects
 - ii. `exists()`: checks if any objects matching QuerySet exist in the database
 - iii. `count()`: retrieves number of objects matching QuerySet without retrieving the actual objects
 - iv. Boolean context: triggers a query to check if any objects exist

3. **In the Exercise, you converted your QuerySet to DataFrame. Now do some research on the advantages and disadvantages of QuerySet and DataFrame, and explain the ways in which DataFrame is better for data processing.**

QuerySet (Advantages):

- a. Integrated with Object-Relational Mapping, making it easy to preform database queries and interact with relational data models defined in Django models
- b. Uses lazy evaluation (does not access database until necessary)
- c. Automatically generates SQL queries based on Django's query syntax

QuerySet (Disadvantages):

- a. Specific to Django and designed primarily for interacting with databases through Django models
- b. Performing complex joins or aggregations across multiple tables can be challenging compared to using SQL directly

DataFrame (Advantages):

- a. Provides a more flexible data structure, making them suitable for a wide range of data processing tasks beyond relational databases
- b. Offers a vast array of functions and methods for data manipulation, cleaning, reshaping, merging, and analysis
- c. Provides a high-level interface that is intuitive and easy to use for common data tasks
- d. Optimized for performance on in-memory data processing

DataFrame (Disadvantages):

- a. Stores data in memory, so handling large datasets that exceed available RAM can be challenging
- b. Has a significant learning curve

DataFrame is better than QuerySet because:

- a. It has a wider range of operations, including data cleaning, manipulation, transformation, analysis, handling missing data, reshaping data, grouping, aggregating, and applying functions across rows or columns.
- b. It performs better when working on in-memory data, providing efficient operations that leverage vectorized operations and optimized algorithms.
- c. It integrates well with other Python libraries used in data science and analysis.

Exercise 2.8: Deploying a Django Project

Coming soon